

FINAL PROJECT

End-to-End ML System Development

Project Guidelines, Topics & Grading Rubrics

Course	DDM501 - AI in Production: From Models to Systems
Duration	4 weeks (Starts Session 5, Presentation at Session 10)
Team Size	3-4 members per team
Weight	40%
Scope	Complete ML system lifecycle: Problem definition to Production deployment

1. PROJECT GUIDELINES

1.1. Objective

The final project is the capstone experience of the DDM501 course. Teams will design, implement, and deploy a complete ML-enabled system that demonstrates all concepts of the course. This project simulates real-world ML engineering, requiring teams to make architectural decisions, implement production-ready code, and present the work.

1.2. Required components

The project includes the following components:

- A. Problem Definition & Requirements (10%)
 - Clear problem statement with business context
 - User requirements and use cases
 - Success metrics (business, system, and model levels)
 - Scope definition and constraints
- B. System Design & Architecture (15%)
 - High-level system architecture diagram
 - Component design with clear responsibilities
 - Data flow diagrams
 - Technology stack justification
 - Trade-offs analysis (scalability, cost, complexity)
- C. Implementation (40%)
 - ML Pipeline:
 - Data ingestion and preprocessing
 - Feature engineering
 - Model training and evaluation
 - Experiment tracking (MLflow or equivalent)
 - Deployment:
 - REST API for model serving
 - Docker containerization
 - Docker Compose for multi-service deployment
 - Monitoring:
 - Prometheus metrics collection
 - Grafana dashboards
 - Alerting rules
- D. Testing & CI/CD (15%)
 - Unit tests for core components
 - Integration tests for API endpoints
 - Data quality tests
 - Model validation tests
 - GitHub Actions CI/CD pipeline
- E. Responsible AI (10%)
 - Fairness analysis and bias detection
 - Model explainability (SHAP, LIME, or equivalent)
 - Data privacy considerations
 - Ethical implications discussion
- F. Documentation (10%)
 - Comprehensive README with setup instructions

- API documentation (Swagger/OpenAPI)
- User guide for deployment and operation

2. PROJECT TOPICS

Topic: Demand Forecasting System

- Description: Predict product demand to optimize inventory management
- ML Approaches: Time series forecasting, LSTM, Prophet, gradient boosting
- Challenges: Seasonality, external factors, multi-step forecasting
- Datasets: Walmart Sales, Store Sales Forecasting, M5 Competition

3. GRADING RUBRICS

The final project is evaluated in two parts: Development (GitHub repo) and Presentation.

3.1. Development rubric (GitHub repo)

3.1.1. Problem definition & requirements (10%)

Criteria	Excellent (9-10)	Good (7-8)	Satisfactory (5-6)	Poor (0-4)
Problem Statement	Clear, specific, well-motivated with business context	Clear problem with some business context	Problem stated but lacks specificity	Vague or missing
Requirements	Complete functional & non-functional requirements with prioritization	Good coverage of requirements	Basic requirements listed	Incomplete
Success Metrics	Clear business, system, and model metrics with targets	Metrics defined at multiple levels	Basic metrics defined	Missing or unclear

3.1.2. System design & architecture (15%)

Criteria	Excellent (9-10)	Good (7-8)	Satisfactory (5-6)	Poor (0-4)
Architecture	Professional diagrams with clear component interactions	Good diagrams with most components	Basic architecture diagram	Missing or unclear
Data Flow	Complete data flow with edge cases handled	Clear data flow documented	Basic data flow	Not documented
Tech Decisions	All decisions justified with trade-off analysis	Most decisions justified	Some justification	No justification

3.1.3. Implementation (40%)

ML Pipeline (15%):

Criteria	Excellent (9-10)	Good (7-8)	Satisfactory (5-6)	Poor (0-4)
Data Pipeline	Robust pipeline with validation, versioning, error handling	Working pipeline with validation	Basic data processing	Incomplete
Model Training	Multiple experiments, hyperparameter tuning, cross-validation	Good training with experiments tracked	Basic training implemented	Minimal effort
Experiment Tracking	Complete MLflow setup with metrics, params, artifacts	Good tracking with most elements	Basic tracking	No tracking

Deployment (15%):

Criteria	Excellent (9-10)	Good (7-8)	Satisfactory (5-6)	Poor (0-4)
API Design	RESTful, well-documented, error handling, versioning	Good API with documentation	Basic working API	Incomplete
Containerization	Optimized Dockerfile, multi-stage builds, security best practices	Working Docker setup	Basic containerization	Not containerized
Orchestration	Complete docker-compose with all services, health checks	Working multi-service setup	Basic compose file	No orchestration

Monitoring (10%):

Criteria	Excellent (9-10)	Good (7-8)	Satisfactory (5-6)	Poor (0-4)
Metrics	Comprehensive system and ML metrics with custom metrics	Good coverage of metrics	Basic metrics	No metrics
Dashboards	Professional Grafana dashboards with meaningful visualizations	Working dashboards	Basic dashboards	No dashboards
Alerting	Meaningful alerts with proper thresholds	Some alerts configured	Basic alerts	No alerting

3.1.4. Testing & CI/CD (15%)

Criteria	Excellent (9-10)	Good (7-8)	Satisfactory (5-6)	Poor (0-4)
Test Coverage	>80% coverage with meaningful tests	70-80% coverage	50-70% coverage	<50% coverage
Test Types	Unit, integration, data quality, model tests	3 types of tests	2 types of tests	Only unit tests
CI/CD Pipeline	Complete CI/CD with linting, testing, building, deployment	Good CI pipeline	Basic CI setup	No CI/CD

3.1.5. Responsible AI (10%)

Criteria	Excellent (9-10)	Good (7-8)	Satisfactory (5-6)	Poor (0-4)
Fairness	Comprehensive bias analysis with	Good fairness analysis	Basic analysis	Not addressed

	mitigation strategies			
Explainability	Multiple explainability methods (SHAP, LIME) implemented	One method implemented well	Basic explanations	No explainability
Ethics	Thoughtful discussion of ethical implications and mitigations	Ethics discussed	Brief mention	Not addressed

3.1.6. Documentation (10%)

Criteria	Excellent (9-10)	Good (7-8)	Satisfactory (5-6)	Poor (0-4)
README	Comprehensive with badges, examples, troubleshooting	Good README with setup guide	Basic README	Minimal/missing
API Docs	Complete OpenAPI spec with examples	Good API documentation	Basic endpoint docs	No API docs
Code Quality	Clean code, type hints, docstrings, consistent style	Good code quality	Readable code	Poor quality

3.2. Presentation rubric

Component	Weight	Excellent	Good	Needs Improvement
Problem & Solution	15%	Clear problem, compelling solution narrative	Good problem-solution flow	Unclear connection
Technical Deep Dive	40%	Expert-level explanation of architecture, implementation, challenges	Good technical coverage	Surface-level explanation
Responsible AI	15%	Thoughtful, comprehensive ethics discussion	Good ethics coverage	Superficial treatment
Q&A Handling	15%	Confident, accurate, detailed responses	Good responses to most questions	Struggled with questions
Live Demo	15%	Smooth, comprehensive demo showing all features	Working demo with minor issues	Demo failed or incomplete

3.3. Individual contribution

Individual grades may be adjusted $\pm 20\%$ based on contribution analysis:

- Git Commit Analysis: Frequency, quality, and distribution of commits
- Individual Q&A: Each member must answer questions about their contribution
- Role Documentation: Clear documentation of individual responsibilities

4. SUBMISSION REQUIREMENTS

4.1. GitHub repository

- Repository must be public or instructor added as collaborator
- All team members must have meaningful commits
- Use meaningful commit messages and proper branching
- Include .gitignore to exclude unnecessary files

4.2. Required files

- README.md: Project overview, setup instructions, usage guide
- ARCHITECTURE.md: System design documentation
- CONTRIBUTING.md: Team member roles and responsibilities
- requirements.txt: Python dependencies
- Dockerfile & docker-compose.yml: Container configurations
- .github/workflows/: CI/CD pipeline configurations

4.3. Presentation

- Duration: 15-20 minutes presentation + 10 minutes Q&A
- Format: PowerPoint/Google Slides/Canva
- Live Demo: All team members must participate